

DMCO: Budget-Aware Co-Optimization of Data Cleaning and AutoML

Xiaou Ding¹ Zekai Qian¹ Siying Chen¹ Hongbin Hu¹ Chen Wang² Hongzhi Wang¹ Jianmin Wang²

Abstract

Data cleaning and automated machine learning (AutoML) are both crucial for reliable learning systems, yet are commonly treated as independent or sequential stages. This separation ignores their strong interaction and leads to inefficient use of limited computational budgets. We propose DMCO, a unified framework that jointly optimizes data cleaning and model construction under a fixed resource budget. DMCO reformulates the traditional two-stage pipeline into a time-sliced process, where data cleaning and AutoML are interleaved and adaptively scheduled. We introduce a gradient-based data cleaning sampling strategy with theoretical guarantees for minimizing gradient estimation variance, and integrates it with loss-driven sampling and progressive AutoML fitting to continuously leverage intermediate data quality improvements. Experiments on six real-world datasets show that DMCO consistently outperforms standalone data cleaning and AutoML baselines on both classification and regression tasks, as measured by F1 score and MSE. Under limited budgets, DMCO achieves up to 82.19% of the performance of full data cleaning with exhaustive AutoML, while remaining robust across different AutoML frameworks.

1. Introduction

The rapid deployment of AI in high-risk sectors like healthcare and finance makes machine learning system reliability a central challenge (Kaur et al., 2022). However, performance is fundamentally constrained by data quality, not just model architecture. Dirty data characterized by missing values or noise induce systematic biases and decision boundary shifts, leading to significant business risks and safety hazards in

¹Harbin Institute of Technology, Harbin, China ²Tsinghua University, Beijing, China. Correspondence to: Hongzhi Wang <wangzh@hit.edu.cn>.

Table 1. Data cleaning and AutoML methods. (For cleaning cost, (1) indicates fully automated, (2) indicates requiring human.)

Target	Representative	Model-Aware	Data Quality	Cleaning Cost
Data	HoloClean Baran	☐ No	✓High	(1)
		☐ No	✓High	(2)
Data	ActiveClean	✓Yes	✓High	(2)
	BoostClean	✓Yes	☐ No cleaned data	(2)
	CPClean	✓Yes	☐ No cleaned data	(1)
Data	Data Transformation	☐ No	☐ Not considered	(1)
	Feature Selection	✓Yes	☐ Not considered	(1) or (2)
	Missing Imputation	☐ No	✓High	(1) or (2)
Model	auto-sklearn	✓Yes	☐ Not considered	(1)
	Auto-WEKA	✓Yes	☐ Not considered	(1)
	Auto-DP	✓Yes	☐ Not considered	(1)

critical applications (Zha et al., 2025; Priestley et al., 2023). For instance, in medical diagnosis, variations in imaging devices across hospitals cause severe performance degradation when models are deployed across institutions (Zech et al., 2018). As discussed in (Whang et al., 2023), machine learning performance is influenced not only by model architecture but critically by data quality, especially under limited resources. Therefore, model improvements should be viewed as the result of the **joint effects of data, models, and resources**. Constraints on computational budgets and cleaning costs dictate feasible strategies, necessitating careful trade-offs between data governance and model optimization.

To address the above challenges, existing research has largely followed two relatively independent directions. First, data governance methods centered on *data quality*, and second, AutoML methods aimed at *model performance*. Table 1 provides a concise comparison of these two categories of methods. Prior work on data governance has mainly focused on **improving data quality through systematic cleaning operations** (Ding et al., 2025b;c). While these approaches have achieved notable progress in error detection and consistency repair, their cleaning decisions are largely guided by predefined rules or distributional assumptions and **do not explicitly account for downstream tasks**. Recent studies have begun to explore **task-aware data cleaning** by incorporating feedback from learning models (Qian et al., 2025). However, under limited computational budgets, exhaustively addressing all data quality issues in a single stage is often inefficient, suggesting that data governance is better

viewed as a *multi-stage process* with dynamically adjusted cleaning priorities (Ding et al., 2025a). In contrast, AutoML focuses on *optimizing model performance given fixed data*. These pipelines typically rely on basic data preprocessing steps and implicitly assume that the input data are already of reasonably high quality, an assumption that often does not hold in real-world scenarios. Thus, data cleaning and model optimization are commonly treated as **two sequential and decoupled stages**, despite being **mutually influential and mutually reinforcing** in practice. Although recent efforts have attempted to integrate cleaning operations into automated pipelines, they largely remain at the level of operator-level integration and heuristic strategies. In practical data governance settings with limited time and resource budgets, this disconnect motivates the need for a unified framework that jointly optimizes data and models under resource constraints.

To enable collaborative decision-making, it is essential to quantify the **influence of individual samples**. For differentiable models, it can be effectively characterized by gradients, which serve as interpretable measures of importance. Specifically, gradients reveal properties: (1) Hard or informative samples typically yield larger gradient norms, significantly impacting model. (2) Redundant samples produce similar gradient directions with limited marginal utility.

Motivated by this, we propose **DMCO (Data–Model Collaborative Optimization)**, a unified framework built upon a family of *differentiable models* that abandons the two-stage pipeline. Instead, the process is divided into multiple time slices. In each slice, DMCO selects the most critical samples for cleaning based on gradient information, while guiding AutoML to prioritize the most suitable differentiable model architecture for the current data. The supported models is summarized in Table 5. Furthermore, we present a concrete task example to compare the effects of different data cleaning and AutoML strategies in Appendix B. It demonstrates that judicious allocation between data cleaning and AutoML can significantly enhance model performance.

The main challenges designing such a co-optimization framework are as follows: (1) The cost of data cleaning scales with data size, while AutoML incurs heavy computational overhead. Under limited budgets, traditional fixed-quota strategies often lead to **resource mismatch**, resulting in either insufficient cleaning or shallow model optimization. (2) Current cleaning strategies typically treat quality issues uniformly, ignoring the model’s heterogeneous sensitivity to errors. Since models do not rely equally on all samples, cleaning based on data statistics risks neglecting **critical samples** for the learning process. (3) Most AutoML methods assume a one-shot, pre-cleaned dataset and cannot perceive intermediate quality improvements. Consequently, they fail to adjust search strategies in real-time, failing to

timely **utilize the positive effects of partial cleaning**.

Addressing these, we introduce DMCO for differentiable models. The main **contributions** are summarized as follows.

(1) We propose a **budget-constrained data–model collaborative optimization framework**. Given the high costs of both data cleaning and AutoML, sequential execution is often infeasible. DMCO optimizes both processes under a **limited time budget**, reformulating the traditional one-shot pipeline into a **progressive, time-sliced decision process**.

(2) DMCO partitions the total budget into TS time slices. Within each slice, a **multi-armed bandit algorithm** allocates resources between cleaning and modeling. Furthermore, DMCO aggregates gradients across a candidate family of differentiable models generated by AutoML, shifting the cleaning cost from expensive counterfactual evaluations to efficient reusable gradient statistics (complexity dominated by $O(Mn)$ aggregation and $O(n \log n)$ selection).

(3) We propose a **loss-based sampling strategy**, reducing the effective training size from n to $k \ll n$, focusing on high-loss samples. Additionally, we introduce a **progressive fitting and search-space pruning strategy**. By inheriting high-quality models, DMCO significantly shrinks the search space, reformulating the one-shot cost $O(R \cdot \text{Train}(n))$ into a more efficient cost $O(\sum_{ts} R'_{ts} \cdot \text{Train}(k_{ts}))$.

(4) We conduct experiments on 6 real datasets, comparing DMCO against 4 baselines. (a) For model performance, DMCO achieves **82.19%** of the performance attained by full data cleaning and exhaustive AutoML, demonstrating that DMCO closely approximates full optimization. (b) Ablation studies validate the effectiveness of the sampling strategies and the time-sliced framework. Moreover, experiments under 3 AutoML frameworks show that DMCO is agnostic to the specific AutoML. (c) Across datasets with 6 error types and 5 corruption levels, results demonstrate that DMCO is robust to diverse error types and magnitudes.

2. Basic Definitions and Problem Formulation

Definition 2.1 (Data Cleaning). Data cleaning is defined as a mapping: $\text{Clean}(D) = D \rightarrow D'$.

We assume that the training set may contain dirty data, while the validation set and the test set are clean. To reduce manual tuning costs, AutoML automatically searches for optimal feature and model combinations under resource budgets, efficiently constructing task-adaptive training pipelines.

Definition 2.2 (Automated Machine Learning). Given a set of hyperparameter optimization algorithms $\mathcal{H} = \{H^{(1)}, \dots, H^{(R)}\}$ and their domains $\Lambda^{(j)}$, a set of feature engineering algorithms $\mathcal{F} = \{F^{(1)}, \dots, F^{(R)}\}$ and their domains $\Psi^{(j)}$, and a loss function $\mathcal{L}(\cdot)$, the automated ma-

chine learning task is to minimize

$$H^*, \lambda^*, F^*, \psi^* = \arg \min_{\substack{H^{(j)} \in \mathcal{H}, \lambda \in \Lambda^{(j)} \\ F^{(j)} \in \mathcal{F}, \psi \in \Psi^{(j)}}} \mathcal{L}(H_{\lambda}^{(j)}, F_{\psi}^{(j)}, D'_{train}, D_{val}).$$

Since cleaning and AutoML incur significant costs, including runtime, I/O, and memory. It is essential to formally define these resource expenditures.

Definition 2.3 (Resource Costs of Data Cleaning and AutoML). The resource consumption for data cleaning and automated machine learning is defined as:

$$Cst_{clean} = Cost(\text{cleaning execution}), \\ Cst_{automl} = Cost(\text{AutoML search}).$$

Specifically, Cst_{clean} accounts for runtime and memory overhead determined by cleaning strategy complexity and data scale, while Cst_{automl} encompasses CPU/GPU resources consumed by feature engineering trials, model structure search, and iterative training and validation.

In resource-constrained environments, data cleaning and AutoML compete for budget yet jointly determine performance. To maximize results, we must optimize cleaning (for data quality) and AutoML (for model selection) simultaneously. Therefore, we formally define the data-model collaborative optimization problem to unify these processes under strict resource constraints.

Definition 2.4 (Data-Model Collaborative Optimization Problem). Let the original training data be D_{train} . After incurring a cleaning cost Cst_{clean} , the data cleaning process maps the data to the cleaned training data

$$D_{train}^{(e)} = \text{Clean}(D_{train}, e),$$

where e denotes the **level of cleaning effectiveness**, characterizing the extent of error removal. While e generally increases monotonically with cleaning cost Cst_{clean} , performance gains vary.

Based on cleaned data $D_{train}^{(e)}$, AutoML searches for model structures and hyperparameters under cost Cst_{automl} , yielding model $M_{D_{train}^{(e)}}$. Given total budget δ , the data-model collaborative optimization problem aims to jointly decide cleaning and AutoML processes to maximize test performance. It is defined as:

$$\max_{e, M} \sum_{t=(x,y) \in D_{test}} \text{metric}(M_{D_{train}^{(e)}}(x), y), \\ \text{s.t. } Cst_{clean} + Cst_{automl} \leq \delta.$$

Definition 2.4 emphasizes that cleaning impacts both resource consumption and model generalization by determining e . The core objective is to balance investments in cleaning and AutoML under budget δ to maximize performance.

3. Co-Optimization Algorithm

3.1. Overview of DMCO

To address the computational bottlenecks of executing full data cleaning and AutoML under fixed time budgets, we propose the DMCO framework. As illustrated in Fig. 1, DMCO employs sampling-based strategies to approximate full-data performance efficiently.

The framework comprises three integrated components. First, **data preparation** utilizes a gradient-based sampling algorithm. By aggregating gradients from multiple differentiable models generated by AutoML, DMCO identifies and cleans high-influence samples that critically impact decision boundaries. Second, the **AutoML component** implements a progressive fitting strategy combined with loss-based sampling. It initializes with meta-learning and progressively prunes the solution space using previous search results. Simultaneously, a loss-based sampling strategy using Gumbel top- k selects representative training subsets, ensuring data diversity while reducing cost. Finally, a **time-slicing strategy** replaces the traditional two-stage pipeline. A multi-armed bandit (MAB) method dynamically allocates the budget between cleaning and modeling. This establishes a closed feedback loop, i.e., data preparation leverages multi-model gradients to target errors, while AutoML benefits from increasingly cleaner data. As time slices progress, this collaborative iteration optimizes both data and models simultaneously, ultimately yielding high-quality outputs.

3.2. Sampling Strategies for Data Cleaning

Within DMCO, data cleaning is redefined not as static pre-processing but as **a dynamic process aimed at improving model stability and generalization**. Since comprehensive cleaning of large datasets is often infeasible due to time and cost constraints, cleaning becomes a limited resource that must be scheduled. Consequently, DMCO focuses on prioritizing samples that yield the maximum performance gain under strict budgets. This requires decisions to go beyond intrinsic data quality and incorporate downstream model states, identifying and cleaning only those specific samples that exert the most significant influence on model.

For a differentiable (or subdifferentiable) model with parameters θ , the per-sample loss is denoted as $\ell_i(\theta) = L(f_{\theta}(x_i), y_i)$, and its gradient (or subgradient)

$$g_i = \nabla_{\theta} \ell_i(\theta) = \nabla_{\theta} L(f_{\theta}(x_i), y_i), \quad (1)$$

describes the magnitude of parameter updates induced by that sample. A larger $\|g_i\|$ typically indicates high fitting error or boundary sensitivity, often corresponding to noisy or critical samples (Aljundi et al., 2019). Consequently, prioritizing high-gradient samples for cleaning yields significant improvements in training trajectories and cost-efficiency.

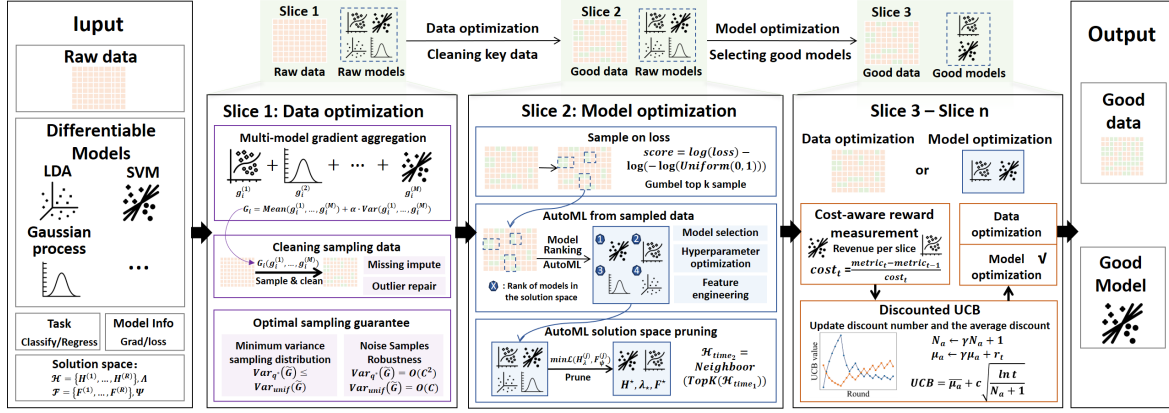


Figure 1. Framework of the proposed DMCO solution.

To mitigate the bias of single-model perspectives, we propose a **gradient-based data cleaning sampling strategy** that leverages the architectural diversity of AutoML. By utilizing intermediate candidate models, we extract gradients from multiple top-ranked predictors to form a robust indicator. The resulting cross-model aggregated gradient score is defined as

$$G_i = \text{Mean}(g_i^{(1)}, \dots, g_i^{(M)}) + \alpha \cdot \text{Var}(g_i^{(1)}, \dots, g_i^{(M)}), \quad (2)$$

where the mean reflects overall sensitivity and the variance captures model disagreement. Samples with high G_i scores are prioritized for cleaning, as detailed in Algorithm 1.

Algorithm 1 Gradient-based Data Cleaning Sampling

Input: data (X, y) , differentiable model family `models`, number of samples k , variance weight α
Output: cleaned dataset D
 Initialize score list $s \leftarrow []$
for $i = 1$ to $|X|$ **do**
 Initialize list $G \leftarrow []$
 for each `model` in `models` **do**
 $g \leftarrow \partial \text{Loss}(\text{model}, x_i, y_i)$
 Append $\|g\|_2$ to G
 end for
 $\mu_i \leftarrow \text{Mean}(G)$, $\sigma_i^2 \leftarrow \text{Var}(G)$, $s_i \leftarrow \mu_i + \alpha \cdot \sigma_i^2$
end for
 Let $\mathcal{I}$ be the indices of the top- k largest values in $\{s_i\}_{i=1}^{|X|}$
 $D \leftarrow \text{Clean}(X[\mathcal{I}])$
return D

Candidate models are provided by AutoML and serve as the input `models` of Algorithm 1. For these candidate models, sample gradients are computed on each model, and the aggregated cross-model gradient score is then calculated. The mean term ensures that gradient information from each candidate model is incorporated, while the variance term favors samples with higher inter-model disagreement, thereby capturing higher information value. These two components

are combined via weighted summation to obtain the final gradient score. Samples are then selected according to the gradient score by choosing the top- k samples, which are subsequently cleaned and returned as the training dataset.

Compared with traditional cleaning methods, DMCO transforms cleaning from a full-data, rule-driven process into a model-aware, incremental process. It not only improves overall data quality during cleaning, but also contributes directly to effective downstream automated machine learning. We provide **theoretical analysis** for the sampling strategy above. Please see it in Appendix C.

3.3. Sampling Strategies for AutoML

Under the time-sliced framework, the reduced duration of each slice precludes exhaustive search on full-scale data. To address this, we design a strategy optimized for limited samples and restricted search spaces.

Loss-driven Weighted Sampling. In AutoML, unrepresentative training samples distort structure evaluation, causing deviation from the globally optimal model. Thus, we introduce a loss-function-based sample importance measure to guide optimization. The core idea is to distinguish *high-impact samples* based on their difficulty. Specifically, we utilize the optimal model from the previous round to compute the loss value. Given the optimal model M_{θ^*} obtained in the previous time slice, according to Definition 2.2, the loss of the model on the training and validation sets can be written as $\mathcal{L}(\theta^*, D'_{\text{train}}, D_{\text{val}})$, which reflects the prediction error of the model on the sample. Samples with larger loss values often lie near class boundaries, pattern transitions, or noise-sensitive regions, and thus exhibit stronger discriminative power for model structure selection. However, sampling solely based on loss ignores diversity, potentially biasing the set toward high-loss outliers and causing AutoML to overfit local hard cases. Thus, we introduce a noisy weighted sampling method without replacement. By incorporating Gumbel noise, this strategy balances loss-based importance

with sample diversity, ensuring the selected subset remains representative while focusing on informative data.

$$s_i = \log \mathcal{L}_i + g_i, \quad g_i \sim \text{Gumbel}(0, 1), \quad (3)$$

Samples with high scores s_i are selected for the AutoML search in the current time slice. This strategy achieves loss-based sampling without replacement, prioritizing high-loss samples while injecting noise to preserve diversity. By preventing budget domination by outliers, it effectively balances importance with representativeness, significantly reducing overfitting and selection bias.

Progressive Fitting of Automated Machine Learning across Time Slices. Loss-based sampling optimizes single-slice costs but fails to reuse knowledge across time. Unlike cleaning, which accumulates benefits, independent AutoML runs on subsets lack continuity and underperform. To address this, we propose a progressive fitting method that treats AutoML as a cumulative process. By preserving key structures and parameters, it enables searches to resume from high-quality solution spaces. This avoids restarts and allows AutoML to progressively refine performance, effectively mirroring the incremental gains of data cleaning.

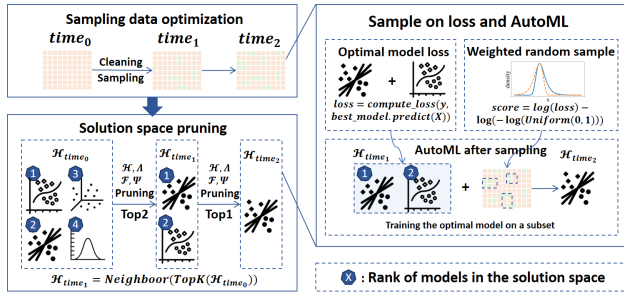


Figure 2. Illustration of the AutoML sampling strategy.

As shown in Fig. 2, progressive fitting optimizes both the model solution space and data quality. Specifically, after the search at time slice, superior model structures and key hyperparameters are preserved. These retained configurations serve as the starting point for the next round, enabling continuous refinement of the solution space.

$$\mathcal{H}_{\text{time}_1} = \text{Neighborhood}(\text{TopK}(\mathcal{H}_{\text{time}_0})). \quad (4)$$

This step effectively shrinks the solution space to a partially explored high-performing region, preventing resource waste on suboptimal candidates. Integrating loss-based sampling with this framework, we design a loss-driven progressive fitting algorithm. The complete procedure and implementation details are presented in Algorithm 2. First, the solution space of AutoML is determined. If this is the first round, a meta-learning algorithm is adopted for fast initialization. If it is not, the better-performing models from the previous AutoML run are used as the solution space for the current

Algorithm 2 Loss-driven Progressive Fitting for AutoML

Input: data (X, y) , current best model `best_model`, pruned model family `pruned_models`, sample size k , iteration index t , mixing weight β
Output: updated `best_model` and `pruned_models`
`models` ← `pruned_models`
 $y_{\text{pred}} \leftarrow \text{best_model.predict}(X)$
 $\ell \leftarrow \text{compute_loss}(y, y_{\text{pred}})$
 $s \leftarrow (1 - \beta) \cdot \text{UniformWeights} + \beta \cdot \ell$
for $i = 1$ **to** $|X|$ **do**
 Sample $u_i \sim \text{Uniform}(0, 1)$
 $g_i \leftarrow -\log(-\log(u_i)) \sim \{\text{Gumbel}(0, 1)\}$
 $\text{score}_i \leftarrow \log(s_i) + g_i$
end for
 Let $\mathcal{I}$ be the indices of the top- k largest values in $\{\text{score}_i\}_{i=1}^{|X|}$ $\{\text{Gumbel-top-}k\}$
`best_model` ← `SelectBestModelAutoML(models)`
`best_model.fit`($X[\mathcal{I}]$, $y[\mathcal{I}]$)
`pruned_models` ← `SelectGoodModelsAutoML(models)`
return `best_model`, `pruned_models`

time slice. After obtaining the solution space, sampling is performed on the data used for AutoML in the current data slice. Specifically, the loss of each data sample is first computed using the optimal model from the previous round, and then the loss is mixed with uniform weights.

3.4. Multi-Armed Bandit-Based Strategy Selection

The traditional two-stage strategy incurs prohibitive costs and suffers from disconnected optimization phases, making it unsuitable for budget-constrained scenarios. To address this, we introduce a **time-slice allocation strategy** that partitions the total budget into TS intervals. As shown in Fig. 3, the system executes a single atomic action (data cleaning, AutoML, or termination) within each slice. By treating updated data states and model parameters as inputs for subsequent iterations, this approach establishes a continuous feedback loop, enabling dynamic co-optimization of data quality and model performance.

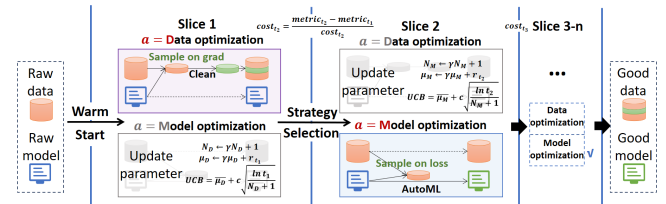


Figure 3. Illustration of the time-slice partitioning strategy.

Let s_{ts} denote the collaborative state at the beginning of time slice ts (including the current data state, the current best model state, and search process information). After

executing action a_{ts} , the system transitions to a new state and enters the next time slice:

$$s_{ts+1} = \mathcal{T}(s_{ts}, a_{ts}), \quad (5)$$

where $\mathcal{T}(\cdot)$ denotes a complete data cleaning or AutoML process that jointly updates data, models, and search space.

If the data cleaning action is selected (e.g., slice 1), the system samples data points based on gradients and performs cleaning, then outputs the cleaned data state together with the (previous-round) AutoML model and search state. If the AutoML action is selected (e.g., slice 2), the system samples data points based on the loss function and performs AutoML search and hyperparameter tuning on the sampled subset to optimize model performance and update the search state.

Based on time-slice partitioning, a **multi-armed bandit (MAB)** serves as the core scheduler, treating data cleaning and AutoML as arms with rewards. In each slice, the system selects an arm and updates its policy based on model performance feedback. This approach dynamically balances the exploitation of high-performing strategies with exploration, preventing long-term bias from early stochasticity.

To ensure comparability of rewards across different time slices and actions, this paper constructs a **cost-aware reward per unit time** as the reward signal:

$$r_{ts} = \frac{\text{metric}_{ts} - \text{metric}_{ts-1}}{\text{cost}_{ts}}. \quad (6)$$

Here, metric_{ts} denotes the model performance after time slice ts , and cost_{ts} denotes the actual time cost consumed in that time slice. Based on the above reward definition, a discounted **Upper Confidence Bound (UCB)** strategy is adopted for decision making. For each arm $a \in \{\text{clean}, \text{AutoML}\}$, discounted trial count N_a , discounted cumulative reward S_a , and discounted mean reward μ_a are maintained:

$$N_a \leftarrow \gamma N_a + 1, S_a \leftarrow \gamma S_a + r_{ts}, \mu_a \leftarrow \frac{S_a}{N_a}. \quad (7)$$

The corresponding upper confidence bound score is defined

$$\text{score}_a(ts) = \mu_a + c \sqrt{\frac{\ln(ts)}{N_a + 1}}, \quad (8)$$

where c is the exploration coefficient. The strategy dynamically balances exploration and exploitation: actions with fewer trials (large confidence intervals) are explored, while those yielding high returns are exploited. To prevent local optima, a stagnation detection mechanism triggers enhanced exploration if performance stalls for H consecutive slices. The adaptive strategy (Algorithm 3) follows a hierarchical logic: prioritizing untried arms, invoking random selection upon stagnation, and otherwise selecting the

highest-scoring arm based on discounted rewards (γ). Unlike traditional one-shot AutoML, DMCO iterates on small-scale, high-value samples. Crucially, it repurposes gradients and loss from AutoML as feedback signals for data cleaning. This transforms AutoML feedback directly into the cleaning loop to optimize both data and model parameters simultaneously.

4. Experiments

4.1. Baseline Methods

The following five methods are designed as baselines: (1) **Dirty Data + Raw Model (DR)**: Neither data cleaning nor model optimization is performed. The model is directly trained on dirty data. (2) **Dirty Data + AutoML (DA)**: Data are not cleaned, but AutoML is applied. AutoML is performed on all data. (3) **Clean Data + Raw Model (CR)**: AutoML is not applied, but data cleaning is performed. Consistent with Dirty Data + AutoML, only part of the data is cleaned under the same cost budget. (4) **Clean Data + AutoML (CA)**: Both data cleaning and AutoML are performed. AutoML is conducted on clean data. (5) **DMCO**: A time allocation strategy is adopted to jointly perform data cleaning and AutoML.

For differentiable models, Auto-sklearn (Feurer et al., 2019) is used with parameter settings. The candidate differentiable models are restricted to the model family listed in Table 5. For the automated machine learning strategy, Auto-sklearn is adopted as the implementation framework in this subsection. The candidate space of Auto-sklearn is restricted to the differentiable model family supported by DMCO.

4.2. Comparative Experiments

We evaluate the effectiveness of DMCO in optimizing downstream models. Experimental results are shown in Table 2. On the Cancer, Nasa, Smartfactory, and Soilmoisture datasets, DMCO consistently demonstrates superior stability and performance advantages across varying task types and noise conditions. From a time-slice perspective, DMCO exhibits high initial performance and approximately monotonic convergence (increasing F1 or decreasing MSE) with minimal oscillation. In most scenarios, DMCO significantly outperforms the baseline (DR), pure AutoML (DA), and pure rule-based cleaning (CR). Notably, it achieves performance levels close to, or even surpassing, the ideal reference upper bound (CA). Given that CA operates on relatively clean data, DMCO’s ability to approach this bound without dataset-specific tuning highlights its robust cross-task generalization.

The comparative analysis reveals the limitations of existing approaches. DA improves over DR by searching model structures but quickly encounters a performance ceiling;

Table 2. Performance comparison of DMCO

Dataset	Method	1	2	3	4	5	6	7	8	9	10
Adult	DR	0.6306	0.6356	0.6380	0.6385	0.6352	0.6374	0.6321	0.6250	0.6289	0.6342
	DA	0.0000	0.6641	0.6210	0.6132	0.6132	0.6146	0.6144	0.6269	0.6248	0.6248
	CR	0.6322	0.6391	0.6415	0.6370	0.6408	0.6338	0.6393	0.6414	0.6403	0.6408
	CA	0.0000	0.5302	0.6527	0.6094	0.6286	0.6286	0.6139	0.6188	0.6139	0.6155
	DMCO	0.0000	0.0000	0.0000	0.6641	0.6641	0.6641	0.6641	0.6081	0.6170	0.6049
Cancer	DR	0.8000	0.8000	0.8000	0.8000	0.8000	0.8000	0.7692	0.8000	0.8000	0.8000
	DA	0.9375	0.9474	0.9247	0.9362	0.9474	0.9362	0.9362	0.9474	0.9474	0.9474
	CR	0.8989	0.8539	0.8791	0.8889	0.8539	0.9011	0.9247	0.9011	0.9592	0.9592
	CA	0.9592	0.9495	0.9495	0.9495	0.9495	0.9495	0.9495	0.9495	0.9592	0.9495
	DMCO	0.9592	0.9691	0.9592	0.9495	0.9592	0.9495	0.9592	0.9592	0.9495	0.9592
Skin	DR	0.8791	0.8791	0.8791	0.9032	0.8791	0.8791	0.8791	0.8667	0.9032	0.8791
	DA	0.9592	0.9592	0.9592	0.9592	0.9592	0.9592	0.9592	0.9592	0.9592	0.9691
	CR	0.9011	0.8791	0.8913	0.8913	0.8913	0.8889	0.8889	0.8889	0.8889	0.9592
	CA	0.9592	0.9495	0.9495	0.9495	0.9495	0.9495	0.9495	0.9592	0.9495	0.9495
	DMCO	0.9592	0.9592	0.9495	0.9691	0.9216	0.9495	0.9495	0.9495	0.9495	0.9495
Smartfactory	DR	0.0222	0.0238	0.0270	0.0286	0.0221	0.0221	0.0221	0.0237	0.0221	0.0221
	DA	0.0000	0.7151	0.7151	0.7179	0.7397	0.7343	0.7343	0.7343	0.7297	0.7427
	CR	0.0221	0.0237	0.0254	0.0205	0.0222	0.0154	0.0239	0.0154	0.0154	0.0154
	CA	0.0000	0.0000	0.0000	0.9275	0.8067	0.9385	0.9384	0.9384	0.9412	0.9417
	DMCO	0.6784	0.6784	0.6768	0.7178	0.8131	0.8803	0.9134	0.9306	0.9454	0.9488
Nasa	DR	34.6802	34.6802	34.6802	34.6802	34.6802	34.6802	34.6802	34.6802	34.6802	34.6802
	DA	20.6762	16.7141	15.1782	12.1341	11.1648	10.8069	10.5492	10.5492	10.5492	9.3171
	CR	29.9315	28.5640	24.1622	22.9370	20.3804	20.1366	18.5961	17.5085	15.6952	13.3376
	CA	11.9683	3.7141	3.7320	3.7320	2.8069	2.8069	2.6210	1.9214	1.9639	1.8630
	DMCO	18.8011	11.9055	11.6682	10.4543	9.1942	9.0713	10.1281	8.2889	6.3613	7.6229
Soilmoisture	DR	2.7162	2.7162	2.7162	2.7162	2.7162	2.7162	2.7162	2.7162	2.7162	2.7162
	DA	0.4298	0.2923	0.4744	0.4491	0.4970	0.4970	0.4970	0.4970	0.4970	0.4380
	CR	2.6490	2.6132	2.4157	2.3583	2.2986	2.1737	2.1119	1.9095	1.8339	1.6847
	CA	0.2490	0.1026	0.0986	0.0750	0.1158	0.1099	0.1248	0.1248	0.1246	0.1246
	DMCO	0.4210	0.4159	0.3636	0.2771	0.2773	0.2274	0.2363	0.1933	0.0815	0.1038

without data quality improvements, it is restricted to local optimization and cannot overcome noise-induced bottlenecks. Conversely, CR relies on static correction rules lacking downstream model feedback, often resulting in negligible gains. On complex datasets like Smartfactory, CR performs near-randomly, confirming that data cleaning unaligned with task objectives is insufficient. In contrast, DMCO does not merely combine these methods but integrates them into a unified, iterative framework. By leveraging a feedback-driven mechanism to adaptively allocate time budgets, DMCO simultaneously optimizes data quality and model parameters. This allows it to rapidly improve performance under limited budgets and sustain superior stability as resources increase. Ultimately, DMCO effectively bridges the gap between data cleaning and automated machine learning, ensuring both strong performance upper bounds and stable lower bounds.

4.3. Ablation Studies

This subsection examines the contribution of each key component in DMCO. Three reduced variants are constructed and compared with the full DMCO: (1) **Data Cleaning Sampling (No loss)**: Loss-function-based sampling is replaced with random sampling to verify the effect of loss-driven sampling. (2) **AutoML Sampling (No grad)**: Gradient-based sampling is replaced with random sampling to verify

the effect of gradient-based sampling. (3) **Time-Slice Allocation Strategy (Two stage)**: The time-slice allocation strategy is replaced with the two-stage strategy to verify the effectiveness of time slicing.

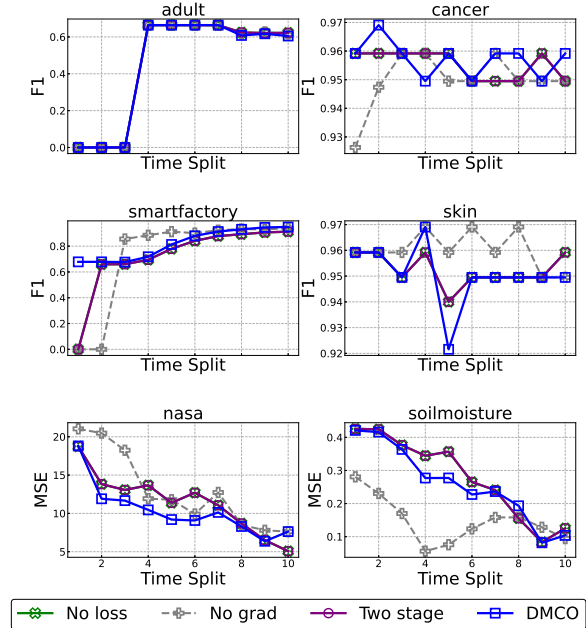


Figure 4. Ablation results of DMCO

As shown in Fig. 4, the complete DMCO consistently outperforms all variants with stable, monotonic convergence across all datasets. While performance gaps are narrower on relatively clean datasets (e.g., Adult, Cancer), DMCO’s advantage becomes significant on noisier tasks (e.g., Nasa, Smartfactory). The “No grad” variant is consistently the weakest, confirming that gradient-based sampling is critical for capturing marginal influence. Similarly, the “Two stage” variant lags due to the severed feedback loop between cleaning and modeling, resulting in slower optimization. By combining gradient information (marginal influence) and loss (sample difficulty), DMCO achieves accurate identification of key samples, avoiding the inefficiency of “No grad” and the blindness of “No loss”. The superior trajectories of the complete framework compared to all reduced variants validate the necessity of integrating these components into a unified, feedback-driven design.

4.4. Robustness Analysis on AutoML Frameworks

To comprehensively evaluate DMCO’s applicability across diverse automated modeling strategies, we introduce AutoGluon and FLAML as additional baselines alongside Auto-sklearn. These frameworks were selected based on reproducibility and representativeness, covering mainstream AutoML paradigms with distinct technical focuses. The detailed information of the frameworks is in Table 6.

In the experimental setup, we replaced the AutoML component within DMCO while aligning resource constraints through native time-budget interfaces (e.g., `time_limit`). Table 3 reports the performance across time slices 1–5. The results demonstrate that DMCO exhibits consistent performance trends across all three frameworks. This confirms that the improvements driven by DMCO are not reliant on a specific search implementation. Instead, the proposed on-demand cleaning strategy offers robust generalizability, capable of delivering stable benefits under varying automated modeling strategies.

For classification tasks, all frameworks achieve comparable optimal performance with DMCO, maintaining high stability on datasets like Cancer and Smartfactory. Notably, Skin benefits significantly from the strong ensemble strategies of Auto-Gluon and FLAML. In regression tasks, distinctions are more pronounced; Auto-Gluon effectively exploits cleaned data for continuous error reduction on Nasa and Soilmoisture, whereas FLAML exhibits limited gains due to its economical search focus.

Overall, these results confirm that DMCO consistently yields stable performance gains across diverse AutoML backends. The observed variations stem primarily from the frameworks’ inherent search and ensemble strategies rather than the effectiveness of the DMCO mechanism itself, validating the strategy’s robustness regardless of the

Table 3. Comparison of DMCO under different AutoML frameworks (AS: Auto-sklearn; AG: Auto-Gluon)

Dataset	AutoML	1	2	3	4	5
Cancer	AS	0.9691	0.9495	0.9495	0.9592	0.9592
	AG	0.9592	0.9592	0.9592	0.9495	0.9495
	FLAML	0.9684	0.9485	0.9293	0.9592	0.9495
Smart factory	AS	0.6784	0.7178	0.8803	0.9306	0.9488
	AG	0.9181	0.9320	0.9323	0.9373	0.9375
	FLAML	0.9236	0.9589	0.9624	0.9605	0.9664
Adult	AS	0.0000	0.6641	0.6641	0.6081	0.6049
	AG	0.6648	0.6658	0.6653	0.6663	0.6660
	FLAML	0.6626	0.5604	0.3655	0.6641	0.6538
Skin	AS	0.9592	0.9691	0.9495	0.9495	0.9495
	AG	0.9988	0.9988	0.9989	0.9990	0.9989
	FLAML	0.9988	0.9990	0.9989	0.9991	0.9991
Nasa	AS	11.9055	10.4543	9.0713	8.2889	7.6229
	AG	7.5643	7.5922	7.1665	6.5391	3.9906
	FLAML	19.6201	20.3156	17.6945	16.3986	15.5613
Soilmoisture	AS	0.4159	0.2771	0.2274	0.1933	0.1038
	AG	0.2702	0.2695	0.2441	0.1766	0.1032
	FLAML	0.5638	0.3660	0.2855	0.1987	0.1687

underlying model search implementation.

4.5. Analysis of Error Types and Error Scales

Experiments are conducted on the Nasa dataset, with error scales set to 10%, 20%, 30%, 40%, 50%. The experimental results are summarized in Table 7 in Appendix F. Overall, DMCO demonstrates consistent advantages across all six error types. As the time-slice limit increases, the MSE generally decreases or converges, indicating that DMCO can continuously mitigate the impact of data defects across scenarios by jointly integrating data cleaning and model training. Notably, even under high noise ratios (40%/50%), DMCO achieves substantial error reduction in later stages and significantly narrows the performance gap between high- and low-noise settings.

Different error types exhibit distinct optimization behaviors. Random errors are generally easier to handle, showing smoother and more monotonic convergence curves, suggesting that DMCO can steadily accumulate improvements under independent noise. In contrast, systematic errors lead to higher error levels and stronger fluctuations, reflecting the difficulty of reducing structured bias. Nevertheless, DMCO is still able to suppress errors in later stages and maintain an overall downward trend. For continuous perturbation noise, the MSE often peaks in early time slices under high noise ratios, but gradually decreases as DMCO progresses, indicating improved robustness over time. From the perspective of error scale, higher noise proportions consistently result in larger errors, slower convergence, and stronger fluctuations across all error types. Despite these challenges, DMCO maintains continuous improvements at all noise levels and

effectively reduces the performance gap between different error scales in later stages, demonstrating its robustness and generalization under complex defect distributions.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgment

This work is supported by National Science and Technology Major Project (2025ZD1607102), the National Natural Science Foundation of China (NSFC) (62572151, 62232005), the National Key Laboratory of Data Space Technology and System (QZQC2026042), the Natural Science Foundation Project of Heilongjiang Province of China (YQ2024F005), and the Heilongjiang Provincial Young Talents Support Program for Science and Technology Innovation (2025QNTJ001).

References

- Aljundi, R., Lin, M., Goujaud, B., and Bengio, Y. Gradient based sample selection for online continual learning. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, pp. 11817–11826, 2019.
- Bhatt, R. and Dhall, A. Skin Segmentation, 2009. URL <https://doi.org/10.24432/C5T30C>.
- Birgelen, A. O. and Niggemann, O. Smart factory: High storage system data for energy optimization. Kaggle, 2018. URL <https://www.kaggle.com/inIT-OWL/high-storage-system-data-for-energy-optimization>.
- Brooks, T. F., Pope, D. S., and Marcolini, M. A. Airfoil self-noise. UCI Machine Learning Repository, 1989. URL <https://doi.org/10.24432/C5VW2C>.
- Chu, X., Ilyas, I. F., and Papotti, P. Holistic data cleaning: Putting violations into context. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, pp. 458–469. IEEE, 2013.
- Ding, X., Wang, H., Li, G., Li, H., Li, Y., and Liu, Y. IoT data cleaning techniques: A survey. *Intell. Converged Networks*, 3(4):325–339, 2022a. doi: 10.23919/ICN.2022.0026. URL <https://doi.org/10.23919/icn.2022.0026>.
- Ding, X., Wang, H., Su, J., Wang, M., Li, J., and Gao, H. Leveraging currency for repairing inconsistent and incomplete data. *IEEE Trans. Knowl. Data Eng.*, 34(3): 1288–1302, 2022b. doi: 10.1109/TKDE.2020.2992456. URL <https://doi.org/10.1109/TKDE.2020.2992456>.
- Ding, X., Song, Y., Wang, H., Wang, C., and Yang, D. MTSClean: Efficient constraint-based cleaning for multi-dimensional time series data. *Proc. VLDB Endow.*, 17(13):4840–4852, 2024. doi: 10.14778/3704965.3704987. URL <https://www.vldb.org/pvldb/vol17/p4840-wang.pdf>.
- Ding, X., Li, G., Tang, Y., Liang, C., Yu, T., Zhou, M., Liu, Y., Qian, Z., Song, Z., and Wang, H. A survey on data asset value change estimation and appreciation with data governance. *Big Data Min. Anal.*, 8(6):1369–1387, 2025a. doi: 10.26599/BDMA.2025.9020034. URL <https://doi.org/10.26599/bdma.2025.9020034>.
- Ding, X., Qian, Z., Wang, H., Chen, S., Tang, Y., Su, H., Hu, H., and Wang, C. UniClean: A scalable data cleaning solution for mixed errors based on unified cleaners and optimized cleaning workflow. *Proc. VLDB Endow.*, 18(11):4117–4130, 2025b. doi: 10.14778/3749646.3749681. URL <https://www.vldb.org/pvldb/vol18/p4117-wang.pdf>.
- Ding, X., Qian, Z., Wang, H., Sun, Z., Chen, S., Su, H., and Hu, H. UniClean: A multi-signal fusion pipeline for optimizing data cleaning workflow. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*, pp. 4600–4603. IEEE, 2025c. doi: 10.1109/ICDE65448.2025.00362. URL <https://doi.org/10.1109/ICDE65448.2025.00362>.
- Doke, A. and Gaikwad, M. Survey on automated machine learning (AutoML) and meta learning. In *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–5. IEEE, 2021.
- Dua, D. and Graff, C. UCI machine learning repository, 2017. URL <http://archive.ics.uci.edu/ml>.
- Feurer, M., Klein, A., Eggenberger, K., Springenberg, J. T., Blum, M., and Hutter, F. Auto-sklearn: Efficient and robust automated machine learning. In *Automated Machine Learning: Methods, Systems, Challenges*, pp. 113–134. Springer, 2019.
- Karlaš, B., Li, P., Wu, R., Gürel, N. M., Chu, X., Wu, W., and Zhang, C. Nearest neighbor classifiers over incomplete information: from certain answers to certain predictions. *Proceedings of the VLDB Endowment*, 14(3): 255–267, 2020.

- Karlaš, B., Salimi, B., and Schelter, S. Navigating data errors in machine learning pipelines: Identify, debug, and learn. In *Companion of the 2025 International Conference on Management of Data*, pp. 813–820, 2025.
- Kaur, D., Uslu, S., Rittichier, K. J., and Durresi, A. Trustworthy artificial intelligence: a review. *ACM computing surveys (CSUR)*, 55(2):1–38, 2022.
- Khayyat, Z., Ilyas, I. F., Jindal, A., Madden, S., Ouzzani, M., Papotti, P., Quiané-Ruiz, J.-A., Tang, N., and Yin, S. BigDancing: A system for big data cleansing. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pp. 1215–1230, 2015.
- Kohavi, R. et al. Scaling up the accuracy of naive-bayes classifiers: A decision-tree hybrid. In *KDD*, volume 96, pp. 202–207, 1996.
- Krishnan, S., Wang, J., Wu, E., Franklin, M. J., and Goldberg, K. ActiveClean: Interactive data cleaning for statistical modeling. *Proceedings of the VLDB Endowment*, 9(12):948–959, 2016.
- Krishnan, S., Franklin, M. J., Goldberg, K., and Wu, E. BoostClean: Automated error detection and repair for machine learning. *arXiv preprint arXiv:1711.01299*, 2017.
- Lessmann, S., Baesens, B., Seow, H.-V., and Thomas, L. C. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European journal of operational research*, 247(1):124–136, 2015.
- Mahdavi, M. and Abedjan, Z. Baran: Effective error correction via a unified context representation and transfer learning. *Proceedings of the VLDB Endowment*, 13(12):1948–1961, 2020.
- Neutatz, F., Chen, B., Alkhatib, Y., Ye, J., and Abedjan, Z. Data cleaning and AutoML: Would an optimizer choose to clean? *Datenbank-Spektrum*, 22(2):121–130, 2022.
- Priestley, M., O’donnell, F., and Simperl, E. A survey of data quality requirements that matter in ml development pipelines. *ACM Journal of Data and Information Quality*, 15(2):1–39, 2023.
- Qian, Z., Ding, X., Wang, C., and Wang, H. Demand-Clean: A multi-objective learning framework for balancing model tolerance to data authenticity and diversity. *Proc. VLDB Endow.*, 18(12):5339–5342, 2025. doi: 10.14778/3750601.3750666. URL <https://www.vldb.org/pvldb/vol18/p5339-wang.pdf>.
- Quemy, A. Two-stage optimization for machine learning workflow. *Information Systems*, 92:101483, 2020.
- Rekatsinas, T., Chu, X., Ilyas, I. F., and Ré, C. HoloClean: holistic data repairs with probabilistic inference. *Proceedings of the VLDB Endowment*, 10(11):1190–1201, 2017.
- Rezig, E. K., Ouzzani, M., Aref, W. G., Elmagarmid, A. K., Mahmood, A. R., and Stonebraker, M. Horizon: Scalable dependency-driven data cleaning. *Proceedings of the VLDB Endowment*, 14(11):2546–2554, 2021.
- Riese, F. M. and Keller, S. Introducing a framework of self-organizing maps for regression of soil moisture with hyperspectral data. In *IGARSS 2018-2018 IEEE International Geoscience and Remote Sensing Symposium*, pp. 6151–6154. IEEE, 2018.
- Salhi, A., Henslee, A. C., Ross, J., Jabour, J., and Dettwiller, I. Data preprocessing using AutoML: A survey. In *2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE)*, pp. 1619–1623. IEEE, 2023.
- Shende, M. K., Feijoo-Lorenzo, A. E., and Bokde, N. D. cleanTS: Automated (AutoML) tool to clean univariate time series at micro-scales. *Neurocomputing*, 500:155–176, 2022.
- Whang, S. E., Roh, Y., Song, H., and Lee, J.-G. Data collection and quality challenges in deep learning: a data-centric ai perspective. *The VLDB Journal*, 32(4):791–813, January 2023. ISSN 1066-8888. doi: 10.1007/s00778-022-00775-9. URL <https://doi.org/10.1007/s00778-022-00775-9>.
- Yakout, M., Berti-Équille, L., and Elmagarmid, A. K. Don’t be scared: use scalable automatic repairing with maximal likelihood and bounded changes. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pp. 553–564, 2013.
- Zech, J. R., Badgeley, M. A., Liu, M., Costa, A. B., Titano, J. J., and Oermann, E. K. Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: a cross-sectional study. *PLoS medicine*, 15(11):e1002683, 2018.
- Zha, D., Bhat, Z. P., Lai, K.-H., Yang, F., Jiang, Z., Zhong, S., and Hu, X. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5):1–42, 2025.
- Zhang, Y., Wang, N., Zeng, Q., and Li, L. Automating data preparation pipeline efficiently via monte carlo tree search. *Information Sciences*, pp. 122730, 2025.

A. Additional Background on Data Governance and AutoML

This appendix provides a detailed discussion of data governance and AutoML, elaborating on their interdependence and complementing the summarized analysis in the main text.

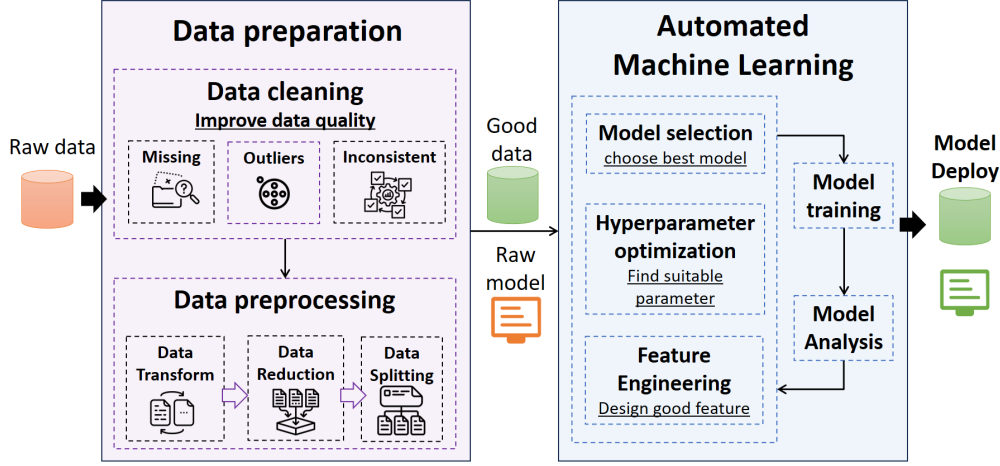


Figure 5. Data preparation and AutoML pipeline.

Traditional data governance approaches primarily focus on improving data quality through systematic cleaning operations, aiming to enhance correctness, completeness, and consistency (Ding et al., 2022b;a). Existing methods can be broadly categorized into rule-driven, data-driven, and hybrid approaches, with representative systems including Holistic (Chu et al., 2013), SCAREd (Yakout et al., 2013), Horizon (Rezig et al., 2021), BigDancing (Khayyat et al., 2015), HoloClean (Rekatsinas et al., 2017), Baran (Mahdavi & Abedjan, 2020), UniClean (Ding et al., 2025b), and MTSClean (Ding et al., 2024). While these methods have achieved notable progress in error detection, consistency repair, and interpretability, their cleaning decisions are largely guided by predefined rules or assumptions about data distributions and **do not explicitly consider downstream tasks or model characteristics**. Consequently, improvements in data quality do not necessarily lead to proportional gains in downstream model performance.

Moreover, **data-centric cleaning methods typically apply uniform strategies across all samples and attributes**, without accounting for the heterogeneous sensitivity of downstream models. The importance of individual samples or fields can vary substantially across tasks, as measured by gradients or Shapley values (Karlaš et al., 2025). For example, in credit risk prediction, missing income values often have limited impact on tree-based models due to surrogate or default splits, whereas even a small number of label errors can significantly distort decision boundaries and degrade predictive performance (Lessmann et al., 2015).

To mitigate these limitations, recent studies have explored **task-aware data cleaning**, where cleaning objectives are constructed by incorporating feedback from learning models. Representative approaches such as ActiveClean (Krishnan et al., 2016), BoostClean (Krishnan et al., 2017), CPClean (Karlaš et al., 2020), and DemandClean (Qian et al., 2025) tightly couple data cleaning with model training or prediction, thereby directly optimizing downstream performance. However, under limited computational budgets, exhaustively addressing all data quality issues in a single stage often fails to yield timely improvements, suggesting that data governance is better formulated as a **multi-stage process** with dynamically adjusted priorities.

AutoML approaches, on the other hand, aim to achieve optimal performance given fixed data through model selection, hyperparameter optimization, and feature engineering. These components are typically optimized within a unified loop of model construction, training, evaluation, and refinement. Although AutoML pipelines include basic data preprocessing steps, such as data transformation, dimensionality reduction, sample reduction, and data partitioning, they generally assume that the input data are already of reasonably high quality. In real-world scenarios, this assumption often does not hold, leading to a disconnect between data optimization and model optimization.

Importantly, **data cleaning and model optimization are not independent processes, but are mutually influential and mutually reinforcing**. Data preprocessing reshapes feature distributions, noise structures, and missingness patterns, thereby affecting the optimal choice of model architectures and hyperparameters. Conversely, model selection, regularization

strength, and learning dynamics determine the model’s sensitivity to different types of data defects.

Despite this interdependence, **most existing systems execute data preparation and model optimization sequentially**. Data cleaning is typically treated as a prerequisite step for automated machine learning (Quemy, 2020; Salhi et al., 2023; Doke & Gaikwad, 2021), rather than being jointly optimized with model selection. For example, in (Salhi et al., 2023; Doke & Gaikwad, 2021), data preprocessing is incorporated as a subroutine of AutoML and executed prior to feature engineering and model selection, while feature engineering and model optimization are iteratively refined. Overall, this design results in a **two-stage pipeline**, where data preparation precedes model optimization without treating data and models as an integrated whole.

More recently, some studies have explored **incorporating data cleaning operations into automated pipeline search or recommendation processes**. For instance, (Neutatz et al., 2022) models data cleaning operations as searchable operators and explicitly considers the trade-off between cleaning cost and performance gains. Building on this idea, Auto-DP (Zhang et al., 2025) further investigates the joint optimization of data cleaning and feature preprocessing strategies within automated modeling pipelines. In addition, cleanTS (Shende et al., 2022) demonstrates the feasibility of jointly searching cleaning operations and model choices in specific time-series scenarios.

Nevertheless, **existing approaches largely remain at the level of operator-level integration and heuristic strategies**. Cleaning policies are typically designed at the column level and uniformly applied to all samples, without characterizing the heterogeneous sensitivity of models to different samples or error types. Moreover, these approaches often assume idealized conditions with sufficient computational resources or negligible costs. In realistic data governance settings, data cleaning and preparation must improve data quality and downstream performance **as much as possible under limited time and resource budgets**, rather than ignoring temporal and computational constraints.

B. An Example for DMCO

To further illustrate the importance of DMCO, we present a concrete task example to compare the effects of different data cleaning and automated machine learning strategies.

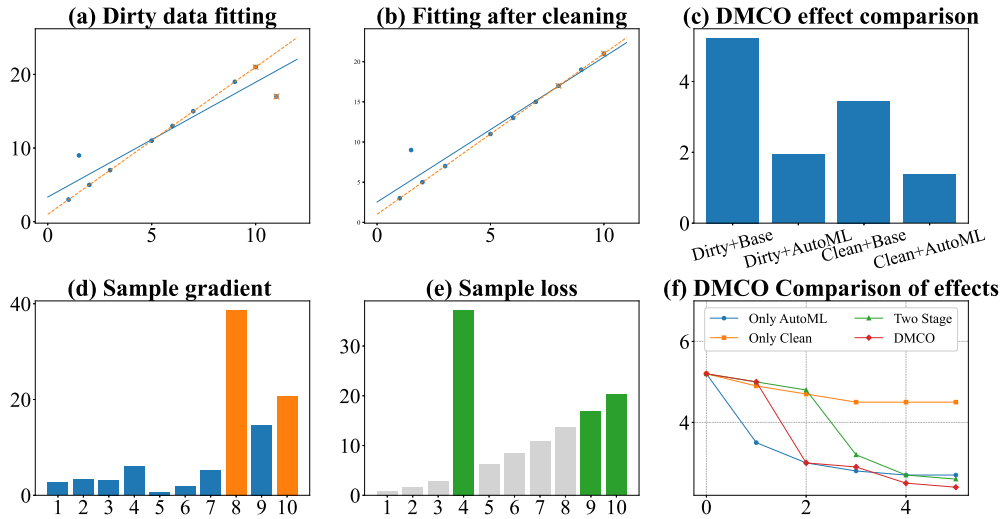


Figure 6. A simple illustrative example of DMCO.

As shown in **Fig. 6(a)**, given a dirty dataset, the crossed blue points represent noisy samples. The true underlying relationship of the clean data is $y = 2 * x + 1$ (orange dashed line). Fitting a linear regression model on the noisy data yields an initial corrupted model $y = 3.4 * x + 1.6$ (blue solid line), which deviates substantially from the true relationship. **Figure 6(b)** shows the model refitted after cleaning samples 8 and 10. Since these samples are critical, their correction leads the learned regression line to better approximate the true function, demonstrating that data cleaning can directly benefit model training. **Figure 6(c)** further illustrates the effect of applying AutoML on top of data cleaning. Compared to data cleaning alone, AutoML yields more pronounced performance gains, and the combination of both achieves the best result.

Figure 6(d) illustrates the loss values of each sample under the dirty model $y = 3.4 * x + 1.6$, where the three samples with the largest losses are marked as high-importance samples. Using the Gumbel top- k strategy, the limited training budget is concentrated on the hardest samples, thereby accelerating model convergence.

As illustrated in **Fig. 6(e)**, gradient norms computed on the dirty model serve as priority scores, identifying Samples 8 and 10 for cleaning. **Fig. 6(b)** shows the refitted model, where the regression line aligns closer to the true relationship. This confirms that targeting high-gradient samples provides direct benefits to model training.

However, the above observations rely on the availability of sufficient resources for full data cleaning and exhaustive AutoML search. **Figure 6(f)** depicts the convergence behaviors of four strategies within short time slices. All curves start from the same point, as they are initialized with the same dirty data and identical weak models. The AutoML-only strategy shows rapid error reduction in the first two time slices, as the model can immediately fit the noisy data. However, since key features remain uncleared, the model quickly encounters a performance ceiling imposed by erroneous data, and subsequent improvements become marginal. The cleaning-only strategy exhibits a different pattern. As erroneous features are gradually corrected, RMSE decreases slightly in the initial steps. However, because the model is never updated, the benefits of cleaning cannot accumulate, and the curve soon reaches a plateau. The two-stage strategy demonstrates even more distinct behavior in the early phase: since the initial time slices are entirely devoted to cleaning, the model is not trained at all, resulting in the same RMSE trajectory as the cleaning-only strategy. Only after the cleaning phase ends does AutoML begin to exploit the cleaned data, leading to noticeable error reductions in later steps. Although this strategy eventually achieves reasonable performance, many early time slices fail to yield effective error reduction, resulting in slow overall convergence.

In contrast, DMCO exhibits a fundamentally different dynamic. After the first cleaning step, RMSE decreases only slightly, similar to the two-stage strategy. However, the subsequent training step immediately leverages the partially cleaned data to update model parameters, yielding a substantial RMSE reduction from 5.0 to 3.0 in the second step, while the two-stage strategy remains around 4.8. The next cleaning step exploits gradients from the updated model, enabling more accurate identification of remaining anomalous samples and further reducing RMSE from 3.0 to 2.9. In the following training step, the model benefits again from cleaner data and more focused samples, reducing RMSE to 2.5. Through multiple iterations, DMCO further lowers the error to 2.4, significantly outperforming the two-stage strategy, which converges at 2.6.

This example demonstrates that, under limited resources, judicious allocation between data cleaning and automated machine learning can efficiently and effectively maximize model performance.

C. Theoretical Analysis of the Sampling Strategy

We first formalize the effect of data cleaning from an optimization perspective.

Let $D = \{(x_i, y_i)\}_{i=1}^n$ denote the training dataset, and $g_i = \nabla_{\theta} \mathcal{L}(f_{\theta}(x_i), y_i)$ denote the (sub)gradient of sample i under the current parameter θ . Under the ideal condition of fully clean data, the expected update direction of model parameters can be represented by the full gradient

$$G^* = \frac{1}{n} \sum_{i=1}^n g_i. \quad (9)$$

In realistic scenarios, only a subset of samples $\mathcal{S} \subseteq \{1, \dots, n\}$ can be cleaned, while uncleared samples still participate in optimization in the form of noisy gradients. The actual gradient update adopted by the model under this condition is

$$\tilde{G}(\mathcal{S}) = \frac{1}{n} \sum_{i \in \mathcal{S}} g_i + \frac{1}{n} \sum_{i \notin \mathcal{S}} \tilde{g}_i, \quad (10)$$

where $\tilde{g}_i$ denotes the perturbed gradient introduced by dirty data.

This work uses the gradient update deviation

$$\Delta(\mathcal{S}) = \mathbb{E} \left[\left\| \tilde{G}(\mathcal{S}) - G^* \right\|^2 \right] \quad (11)$$

as a measure of cleaning effectiveness, characterizing the extent to which cleaning reduces optimization perturbations.

From the above definitions, under limited cleaning budgets where only a subset of samples can be repaired, the goal of data

cleaning can be formalized as

$$\min_{\mathcal{S}: |\mathcal{S}| \leq k} \Delta(\mathcal{S}), \quad (12)$$

i.e., selecting a subset $\mathcal{S}$ of at most k samples such that the actual gradient update is, in expectation, as close as possible to the ideal full update G^* .

Noting that $\Delta(\mathcal{S})$ essentially corresponds to the variance of gradient estimation error, the above problem is equivalent to minimizing the variance of gradient estimation under unbiasedness constraints. Therefore, under limited budgets, selecting samples for cleaning can be transformed into an importance sampling problem whose objective is to obtain the most stable and closest-to-ideal update direction under limited intervention.

Under limited budgets where only a small number of samples can be cleaned, prioritizing samples with large gradients for cleaning is equivalent to adopting minimum-variance importance sampling to approximate the ideal full update, thereby reducing stochastic fluctuations in training updates and making optimization trajectories closer to those under full data in expectation.

Let data points be sampled according to distribution $q = \{q_i\}_{i=1}^n$ and used for parameter updates, and define

$$\tilde{G} = \frac{1}{n} \sum_{i \in \mathcal{S}} \frac{g_i}{q_i} \quad (13)$$

as the unbiased estimator constructed from sampled results (importance weighting ensures $\mathbb{E}[\tilde{G}] = G$). Under this framework, cleaning and training can be jointly understood as an update process under gradient estimation error. If the sampling variance is large, i.e., $\tilde{G}$ fluctuates significantly across training rounds, the optimization trajectory deviates from the true gradient direction, leading to slower or unstable convergence. The core theoretical problem can thus be formalized as

$$\min_q \text{Var}_q(\tilde{G}) \quad \text{s.t.} \quad \mathbb{E}[\tilde{G}] = G, \quad \sum_i q_i = 1. \quad (14)$$

Here, the standard vector variance definition $\text{Var}_q(\tilde{G}) = \mathbb{E}[\|\tilde{G} - G\|^2]$ is adopted.

Theorem C.1 (Minimum-Variance Sampling Distribution). *When the sampling probability satisfies $q_i^* \propto \|g_i\|$, the variance of the gradient estimator $\tilde{G}$ is minimized, moreover, this distribution strictly outperforms uniform sampling:*

$$\text{Var}_{q^*}(\tilde{G}) \leq \text{Var}_{\text{unif}}(\tilde{G}). \quad (15)$$

Proof. The variance of the unbiased estimator can be written as

$$\text{Var}_q(\tilde{G}) = \frac{1}{n^2} \sum_{i=1}^n \frac{\|g_i\|^2}{q_i} - \|G\|^2. \quad (16)$$

Under the constraint $\sum_i q_i = 1$, let $u_i = \sqrt{q_i}$ and $v_i = \|g_i\|/\sqrt{q_i}$. By the Cauchy-Schwarz inequality,

$$\left(\sum_i q_i \right) \left(\sum_i \frac{\|g_i\|^2}{q_i} \right) \geq \left(\sum_i \|g_i\| \right)^2,$$

with equality if and only if $q_i \propto \|g_i\|$. Substituting into Eq. (16) completes the proof. $\square$

Theorem C.1 shows that if only a small number of samples can be used to approximate the full update, importance sampling according to $\|g_i\|$ minimizes the variance of $\tilde{G}$, making each update direction more stable and closer to the full update G . The essence of cleaning is to reduce the perturbation caused by erroneous samples. High-gradient samples contribute most to the update direction; if they contain noise, missing values, or label inconsistencies, their gradient deviations are injected into optimization with higher weight, causing significant deviation in update directions. Thus, **prioritizing high-gradient samples for cleaning is equivalent to prioritizing the elimination of the most dominant sources of perturbation**. These samples lie near decision boundaries or correspond to anomalous losses, and thus determine classification boundaries and training stability. Allocating limited budgets to such samples maximizes the denoising effect of training trajectories per unit cost.

Based on this, the improvement of gradient-based sampling over uniform sampling can be further quantified.

Corollary C.2 (Variance Reduction). *The variance advantage of optimal sampling over uniform sampling can be expressed as the variance of gradient norms:*

$$\Delta \text{Var} = \text{Var}_{\text{unif}}(\tilde{G}) - \text{Var}_{q^*}(\tilde{G}) = \text{Var}(\|g\|). \quad (17)$$

This result indicates that in scenarios with larger gradient disparities among samples (often corresponding to higher error rates, label inconsistencies, or feature noise), gradient-based importance sampling yields more significant variance reduction; in other words, **the dirtier the data, the greater the relative advantage of this strategy**.

Corollary C.3 (Robustness under Noisy Samples). *If there exist outlier samples with gradient ratio $C \gg 1$, then*

$$\text{Var}_{\text{unif}} = O(C^2), \quad \text{Var}_{q^*} = O(C),$$

i.e., the variance growth order under gradient-based sampling is lower than that under uniform sampling.

This corollary further shows that when outlier samples have low frequency but dominate gradient magnitudes, uniform sampling suffers from quadratic error amplification, whereas gradient-driven sampling exhibits superior robustness, making it particularly suitable for scenarios with significant noise and limited cleaning resources.

In summary, starting from the minimum-variance theory of gradient estimation, this section proves that sample selection based on gradient norms possesses optimal variance properties under unbiasedness constraints, and further explains its correspondence with practical cleaning processes. **High-gradient samples are both key contributors to parameter updates and the most critical perturbation sources under dirty data.** Therefore, prioritizing high-gradient samples for cleaning under limited budgets theoretically guarantees updates closer to full-data updates and, in practice, improves training stability and collaborative optimization efficiency, providing rigorous theoretical support for the gradient-driven cleaning strategy proposed in the previous section.

D. Algorithms

For the time slice strategy in Section 3.4, we designed an adaptive strategy selection algorithm that uses a multi-armed slot machine to make choices between data cleaning and automatic machine learning, thereby optimizing the final result.

E. Experiment details

This section provides details of the experimental section that were not shown due to space limitations.

E.1. Datasets

The following datasets are used in the experiments:

Adult (Kohavi et al., 1996): Contains data from the 1994 U.S. Census.

Cancer (Dua & Graff, 2017): A breast cancer diagnosis dataset used to predict whether a tumor is benign or malignant.

Nasa (Brooks et al., 1989): Provided by NASA, including typical spacecraft sensor data and fault detection data.

Factory (Birgelen & Niggemann, 2018): Records production operation data from smart factories, used for analyzing production efficiency, detecting anomalies, or optimizing processes. The features include temperature, pressure, machine operating status, and workpiece processing time.

Soilmoisture (Riese & Keller, 2018): Time-series data recording soil moisture.

Skin (Bhatt & Dhall, 2009): Contains skin texture data.

Detailed information of the above datasets is summarized in Table 4.

Algorithm 3 Adaptive Strategy Selection Mechanism

Input: number of time steps TS , exploration coefficient c , scaling factor β , discount factor γ , stagnation threshold H
Output: action sequence $\{a_{ts}\}_{ts=1}^{TS}$
 $\mathcal{A} \leftarrow \{\text{clean}, \text{AutoML}\}$
for $a \in \mathcal{A}$ **do**
 $N_a \leftarrow 0, S_a \leftarrow 0, \mu_a \leftarrow 0$
end for
 $bestMetric \leftarrow -\infty, stagnationCount \leftarrow 0$
Initialize $metric_0$
for $ts = 1$ **to** TS **do**
 if $\exists a \in \mathcal{A}$ s.t. $N_a = 0$ **then**
 Choose any untried arm and set $a_{ts} \leftarrow a$
 else if $stagnationCount \geq H$ **then**
 Choose a random arm and set $a_{ts} \leftarrow a, stagnationCount \leftarrow 0$
 else
 $a_{ts} \leftarrow \arg \max_{a \in \mathcal{A}} \left(\mu_a + c \sqrt{\frac{\ln(ts)}{N_a + 1}} \right)$
 end if
Execute a_{ts} and obtain $metric_{ts}$ and $cost_{ts}$
 $r_{ts} \leftarrow \frac{metric_{ts} - metric_{ts-1}}{cost_{ts}}, N_{a_{ts}} \leftarrow \gamma N_{a_{ts}} + 1, S_{a_{ts}} \leftarrow \gamma S_{a_{ts}} + r_{ts}, \mu_{a_{ts}} \leftarrow \frac{S_{a_{ts}}}{N_{a_{ts}}}$
 if $metric_{ts} > bestMetric$ **then**
 $bestMetric \leftarrow \beta \cdot metric_{ts}, stagnationCount \leftarrow 0$
 else
 $stagnationCount \leftarrow stagnationCount + 1$
 end if
end for
return $\{a_{ts}\}_{ts=1}^{TS}$

E.2. Evaluation Metrics

Algorithm performance is mainly evaluated from the perspective of downstream model effectiveness. F1 score and MSE are used to measure downstream model performance.

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall} \quad (18)$$

A higher F1 score indicates better classification performance, where $precision = \frac{TP}{TP+FP}$, $recall = \frac{TP}{TP+FN}$, TP denotes true positive samples, FP denotes false positive samples, and FN denotes false negative samples.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i^* - y_i')^2, y_i \in t_i \in D_{test} \quad (19)$$

MSE measures the distance between predicted values and ground truth in regression tasks. Lower MSE indicates better predictive performance.

E.3. Additional Experimental Results

To better illustrate the effects of each method in the comparative experiment in Section 4.2, we have drawn a line graph, as shown in Figure 7.

F. Supplemental Tables

DMCO limits its research scope to a class of differentiable models with a well-defined loss function structure and parameters that can be updated via gradients. This class of models not only covers mainstream linear models, discriminative models,

Table 4. Dataset information

Dataset	Tuples	Attributes	Task	Label
Adult	45,223	15	C	Income
Cancer	700	12	C	Class
Nasa	1,504	6	R	SPL
Factory	23,645	19	C	Labels
Soilmoisture	679	129	R	Soil Moisture
Skin	245057	3	C	Label

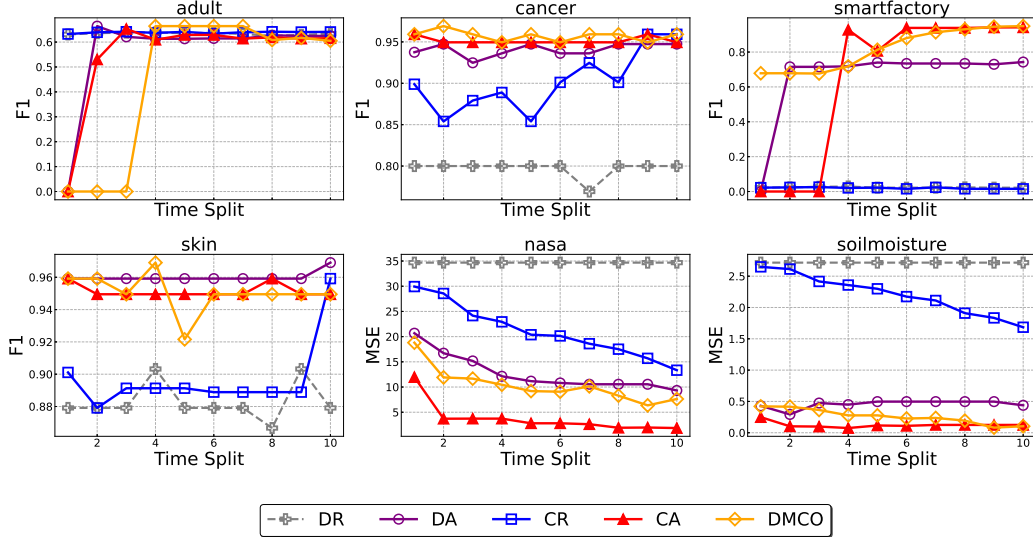


Figure 7. Performance comparison of DMCO

and neural networks, but also includes several probabilistic generative models with smooth likelihood functions, whose optimization process can be stably solved in a continuous space. Given a defined model hypothesis space, performance metrics, sample sensitivity estimation, and resource allocation strategies in collaborative optimization problems can all be consistently characterized theoretically, thus ensuring the overall method’s analyzability and interpretability. The family of differentiable models supported and discussed in this paper is shown in Table 5.

Table 5. Summary of differentiable models supported by DMCO (Hinge: used for large-margin classification, penalizing insufficient margins or misclassified samples; Squared: used for regression, measuring squared prediction error; CE: cross-entropy for probabilistic classification; NLL: negative log-likelihood; log-likelihood: log likelihood).

Model	Differentiability	Common Loss	Task	Strengths
SVM / SVR	Subdifferentiable	Hinge / Squared	C / R	Robust to outliers
SGD	Differentiable	CE / Hinge	C / R	Suitable for large-scale sparse data
MLP	Differentiable	CE	C / R	Captures nonlinear features
GaussianNB	Differentiable	NLL	C	Fast training, strong small-sample performance
LDA	Differentiable	NLL	C	Linear discrimination, high interpretability
QDA	Differentiable	NLL	C	Class-specific covariance modeling
ARD Regression	Differentiable	NLL	R	Strong feature selection capability
Gaussian Process	Differentiable	Log-likelihood	R	Uncertainty quantification, non-parametric modeling

To more comprehensively evaluate the applicability of DMCO, we introduce three frameworks in Table 6. Auto-sklearn focuses on pipeline-based algorithm selection and hyperparameter optimization combined with ensembling; Auto-Gluon emphasizes engineering-oriented multi-model training and Bagging/Stacking; FLAML focuses on time-budget-driven economical search to approach strong performance with lower training cost.

This section presents a comparison of specific model performance on the NASA dataset, addressing the impact of error

Table 6. AutoML frameworks adopted in DMCO

Tool	Core Mechanism	Resource / Cost Control
Auto-sklearn	scikit-learn pipeline-based search; Bayesian optimization as the core, combined with meta-learning for warm start, automatically performing algorithm selection and hyperparameter optimization	Provides global search budget and per-run training budget (e.g., <code>time_left_for_this_task</code> , <code>per_run_time_limit</code>); controllable ensemble size (<code>ensemble_size</code>)
Auto-Gluon	Engineering-oriented AutoML emphasizing multi-model training and systematic ensembling via Bagging and multi-layer Stacking, achieving strong baselines with limited heavy HPO	Commonly uses <code>time_limit</code> to constrain training budgets and provides different presets to balance accuracy and training/inference cost
FLAML	Cost-efficient AutoML targeting economical search and tuning under fixed time budgets, emphasizing strong performance with minimal training cost	Core parameters include <code>time_budget</code> (total budget) and <code>max_iter</code> (iteration count), supporting on-demand sampling and incremental search to reduce cost

types and sizes discussed in Section 4.5 on DMCO.

Table 7. DMCO results under different error types and error scales (metric: MSE)

Error	Slice	10%	20%	30%	40%	50%
Gauss	1	4.6341	7.2138	10.1891	9.3492	15.4330
	2	3.8893	5.8780	9.2061	9.5529	10.5314
	3	3.0908	3.7570	7.2423	10.2543	9.0082
	4	2.5531	3.4039	7.2293	10.9479	7.1522
	5	2.1293	2.4698	2.6078	6.5634	5.7788
Random Missing	1	2.9694	3.6114	3.7712	4.2116	4.6574
	2	2.9924	3.1956	3.3939	3.6291	3.7571
	3	2.7198	2.8409	3.0509	3.3173	2.5408
	4	1.8259	1.9772	1.9998	2.4298	2.3850
	5	1.8140	1.9279	1.9464	2.0648	2.1285
Random Outliers	1	2.8996	3.4190	3.6808	3.5039	4.3221
	2	2.8588	3.0037	3.0616	2.7742	2.9744
	3	2.4001	2.4962	2.3747	2.5813	2.7228
	4	1.8424	1.9882	1.9364	2.1091	2.0485
	5	1.5420	1.6601	1.9317	1.9920	1.7777
System Missing	1	4.8507	6.0904	6.9107	7.9315	10.2822
	2	3.8952	4.9574	5.2762	7.4432	5.9839
	3	4.2782	5.6087	7.7585	8.1804	8.2794
	4	3.8203	4.5178	8.7057	6.8503	6.5453
	5	2.7546	4.2538	6.3301	4.7703	4.5725
System Outliers	1	4.5709	5.5496	6.4436	8.1070	9.8742
	2	3.8257	4.3738	5.3324	6.4531	8.5719
	3	3.6903	4.8766	9.0154	8.5371	9.9474
	4	3.0792	5.7400	6.1939	7.7183	8.9822
	5	2.3672	2.8149	3.8013	5.0070	5.1331
White	1	4.8524	7.3013	11.0164	8.6517	11.9903
	2	4.3633	5.9335	8.3455	11.2134	15.8414
	3	3.1698	4.3673	7.2693	9.0108	9.1872
	4	2.7766	4.1147	6.8037	8.3676	8.3430
	5	2.2870	2.6600	4.1542	5.1804	6.6911